

无96 吴昊宸 2019011126

摘要：

现今，科学的力量正在逐步介入金融领域，随机过程的数学基础在股价预测上的引用，就是一个典型的例子。基于高斯过程的回归分析为非线性的回归问题提供了一种性质有效的解决办法。

在本文中，我借助高斯过程回归(Gaussian Process Regression,GPR)，使用MATLAB语言，对一年之内的股票市场进行了预测和验证，在github上开源链接如下：

[skywind2002/RandomProcess-GPR: 大三上《概率论与随机过程\(2\)》大作业——高斯过程回归 \(github.com\)](#)。就创新点和亮点而言，本文的工作在使用长期GPR预测碰壁之后，将目光转向短期GPR预测。通过不断迭代训练集的方法，解决了GPR在时间过长的情况下，核函数退化的问题，获得了不错的改进成果。同时，利用短期GPR，还可以在牺牲一些时效性的情况下，将预测准确度提升到接近完美的水平。

在结构上，本文首先分析问题背景和GPR的数学基础，之后详细介绍GPR在我的代码中的实现过程与细节。最后，将得到的实验结果和国外学生的Project作对比，分析自己方法的优势和特点。

关键词：高斯过程回归 股价预测

1.数学基础

- 高斯过程回归模型

回归分析是非常重要的统计分析领域。对于线性回归问题，从数学上可以证明最小二乘法具有最优性。而对于非线性问题，我们可以通过核函数(kernel)将其转变为线性问题进行求解。

高斯过程回归是一种针对非线性回归问题行之有效的解决方法。参考[1]以及[2]中的讲解，在此可以详细解释GPR如下：

我们首先假设回归函数的输出符合高斯过程 $f(X) \sim N(m(X), K(X, X'))$ 。其中 $m(X) = E[f(X)]$ 为均值(本文对数据做了归一化处理, 因此均值为0向量)。而协方差矩阵 K 的表示形式如下:

$$K = \begin{bmatrix} k(x_1, x_1; \theta) & k(x_1, x_2; \theta) & \cdots & k(x_1, x_n; \theta) \\ k(x_2, x_1; \theta) & k(x_2, x_2; \theta) & \cdots & k(x_2, x_n; \theta) \\ \vdots & \vdots & \ddots & \vdots \\ k(x_n, x_1; \theta) & k(x_n, x_2; \theta) & \cdots & k(x_n, x_n; \theta) \end{bmatrix}$$

其中 $k(x_i, x_j; \theta)$ 被称作“核函数”(kernel function), 有着可以变化(学习)的参数 θ 。核函数决定了高斯回归模型的性能, 有着举足轻重的作用。总体上, 核函数需要遵守着 x_i 和 x_j 相距越远, 输出的值就越小的原则。

在利用高斯过程对输入 X 真实的输出 Y 完成建模之前, 还需要考虑噪声的因素(建模为0均值高斯噪声), 也就是说, Y 是一个高斯过程加上一个白噪声,
 $Y = f(X) + N(0, \sigma^2 I_n)$ 。

假设参数已经固定(参数优化的步骤见下), GPR的任务就是根据训练数据, 包括已知的训练输入值 $\mathbf{x} = [x_1, x_2, \cdots, x_n]$ 和已知的训练输出值 $\mathbf{y} = [y_1, y_2, \cdots, y_n]$, 以及未知对应输出的测试输入值 $\mathbf{x}^* = [x_1^*, x_2^*, \cdots, x_m^*]$ 来对测试输出值 $\mathbf{y}^* = [y_1^*, y_2^*, \cdots, y_m^*]$ 进行预测。由高斯过程的性质, 任意个时间采样得到的随机变量均符合联合高斯分布, 即:

$$\begin{bmatrix} y \\ f(x^*) \end{bmatrix} \sim N(\mathbf{0}, \begin{bmatrix} K + \sigma^2 I_n & (K^*)^T \\ K^* & K^{**} \end{bmatrix})$$

其中 K^* 和 K^{**} 与 K 有着一样的定义, 分别是 $\mathbf{x}^*$ 与 $\mathbf{x}$ 的互协方差矩阵, 与 $\mathbf{x}^*$ 的自协方差矩阵。那么, 由条件高斯分布, 在知道 y 的先验分布为 $N(0, K)$ 的情况下, $f(x^*)$ 的后验分布即为:

$$f(x^*)|y \sim N(K * (K + \sigma^2 I_n)^{-1} y, K^{**} - K * (K + \sigma^2 I_n)^{-1} (K^*)^T)$$

再加上噪声, 我们就得到了 y^* 的后验分布, 从而完成了根据 $\mathbf{x}, \mathbf{y}$ 和 $\mathbf{x}^*$ 完成对 $\mathbf{y}^*$ 的预测:

$$y^*|y \sim N(K * (K + \sigma^2 I_n)^{-1} y, K^{**} - K * (K + \sigma^2 I_n)^{-1} (K^*)^T)$$

• 核函数参数优化

上面已经将高斯过程回归的原理阐释的颇为详尽, 但是却没有提到GPR中最重要的部分——核函数。事实上, 核函数反映了我们用于建模的高斯过程的协方差矩阵随着时间增长的演化过程。不恰当的核函数选取会导致GPR偏离全局最优解, 严重的甚至会导致问题变得病态且奇异, 从而难以使用数值方法求解。

因此，在对下载的数据进行GPR预测之前，我们需要针对核函数中的参数进行优化求解(具体核函数的选取在“程序细节”一节中将详细阐释)。

对核函数参数 θ 的优化，我们可以采取监督学习的方式，以 $(\mathbf{x}, \mathbf{y})$ ，也就是训练数据集为基础，采取对数似然函数如下(具体见[2]23页)：

$$-\log P(y|X, \theta) = \frac{1}{2} \log |K + \sigma^2 I_n| + \frac{1}{2} \mathbf{y}^T (K + \sigma^2 I_n)^{-1} \mathbf{y} + \text{const}$$

最小化这个函数(注意优化的是负对数)，我们就可以得到最佳的核函数参数 θ 。

2.程序算法细节

根据上文所说明的高斯过程的原理，我们可以发现GPR的结果只取决于核函数的选取以及核函数参数的优化方式，其他都是非常平凡的运算工作。因此，下面我将对这两个关键点，结合代码充分分析我的实现方式。

- 核函数选取

在核函数的选取上，[3]中选取了"Squared Exponential Covariance","Matern Class Covariance","Rational Quadratic Covariance","Marginal Likelihood Gradient"四种核函数。而我所选择的核函数与[3]有所不同，参考了我在自行查阅文献的时候找到的[4]以及github网站教程[Gaussian processes \(3/3\) - exploring kernels \(peterroelants.github.io\)](https://peterroelants.github.io)。最终，在筛选下，我在程序中一共实现了3种核函数：

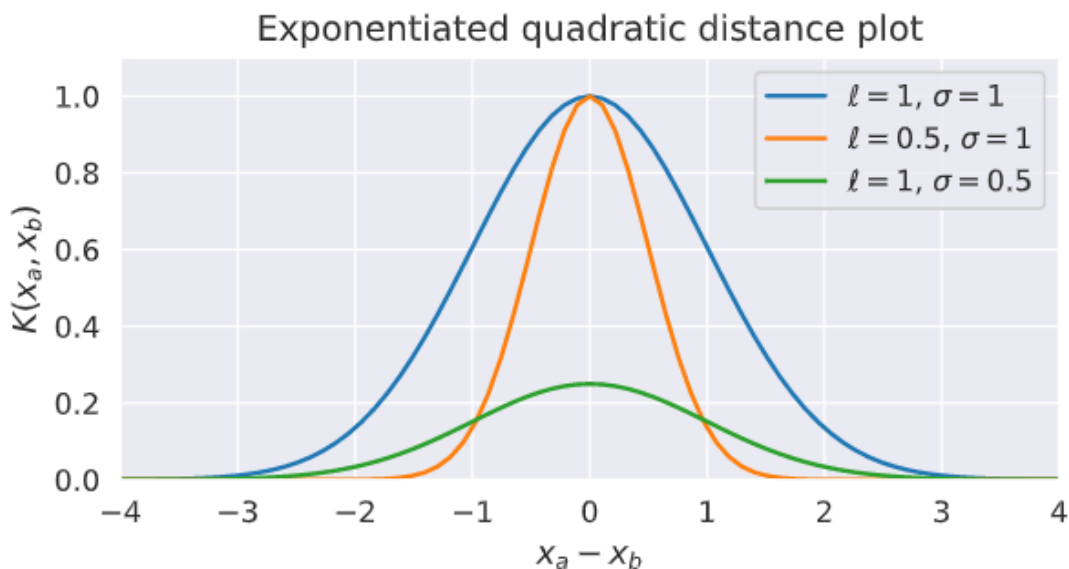
1. Exponentiated quadratic kernel(平方指数核函数,eqk)

数学形式为： $k(x_a, x_b) = \sigma^2 \times \exp(-\frac{\|x_a - x_b\|^2}{2l^2})$ 。

这是一个以L2范数为基础的核函数(与高斯分布类似)，随着两个输入向量之间的L2范数越大，核函数成指数级别的减小，在 $x_a = x_b$ 时，核函数达到最大值。

在这个核函数中，能够被优化的参数有两个， l 和 σ 。

这个核函数可视化如下：



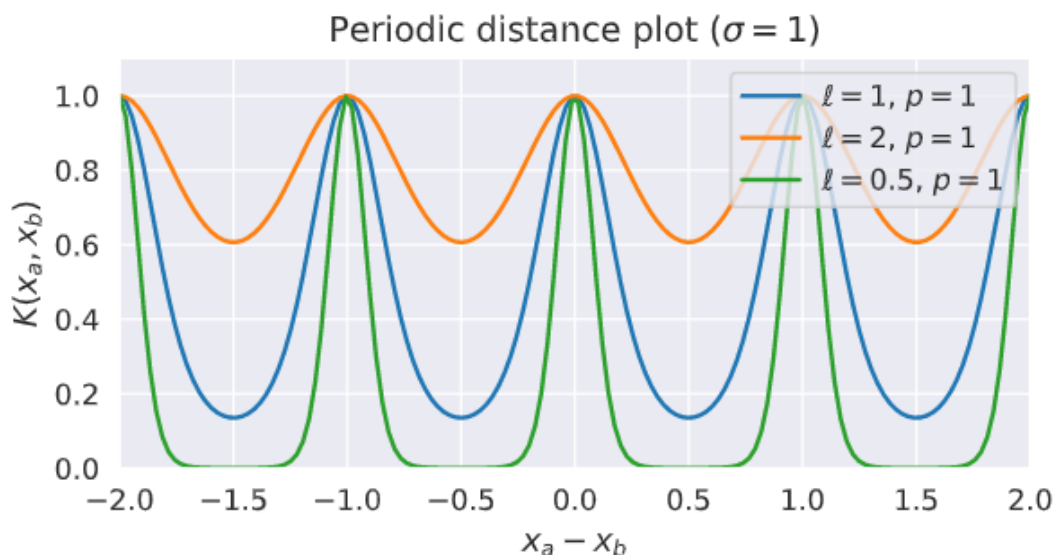
2. Periodic kernel(周期核函数,pk)

数学形式为 $k(x_a, x_b) = \sigma^2 \times \exp(-\frac{2}{l^2} \sin^2(\pi \frac{|x_a - x_b|}{p}))$ 。

这是一个以L1范数为基础的核函数。在指数上引入的sin函数提高了我们对周期函数的表达能力。但是对于非周期函数可能并没有很好的作用。

在这个核函数中，能够被优化的参数有三个， l , p 和 σ 。

这个核函数可视化如下：



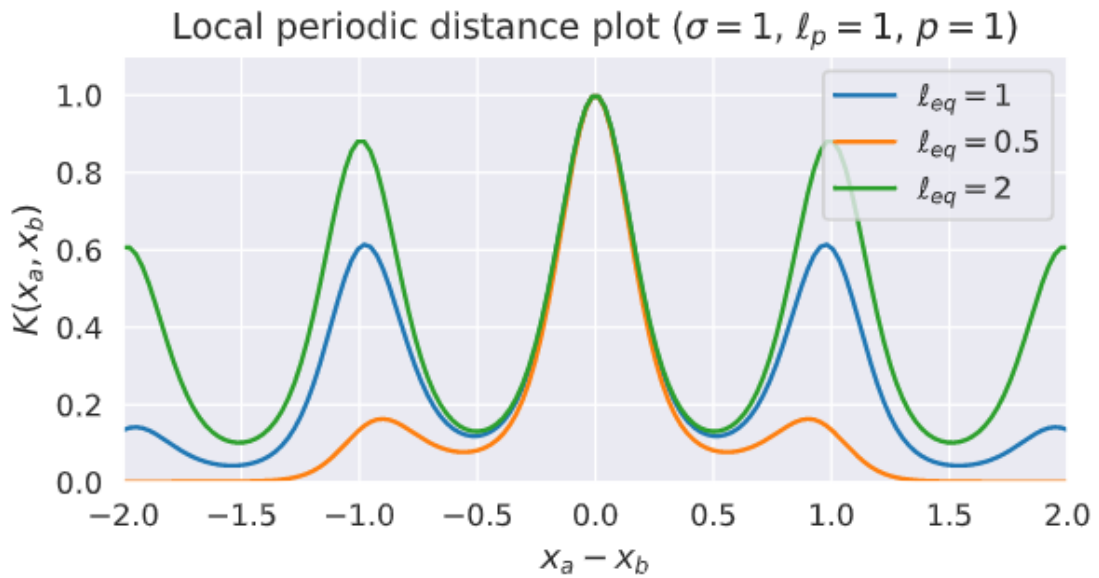
3. Local periodic kernel(混合周期核函数,lpk)

数学形式为 $k(x_a, x_b) = \sigma^2 \exp(-\frac{2}{l^2} \sin^2(\pi \frac{|x_a - x_b|}{p})) \exp(-\frac{\|x_a - x_b\|^2}{2l_{eq}^2})$ 。

这是一个混合了L1与L2范数的核函数。相比于之前的周期核函数，允许周期在价赔偿的距离内进行变化。在大多数情况下有着比周期核函数更好的性能。

在这个核函数中，能够被优化的参数有4个， l_p, p, l_{eq} 和 σ 。

这个核函数可视化如下：



这三种核函数由KernelFunc.m中的KernelFunc函数统一实现。这个函数跟根据输入的两个向量，参数列表以及核函数名称计算核函数的输出，在参数列表出错的时候会抛出错误提示并且结束整个程序。其核心代码和注释如下：

```
function result = KernelFunc(x1, x2, param, name)
% 核函数计算,name指示不同的核函数类型
% 函数会根据name和超参数param计算出向量x1和向量x2之间的协方差
% 具体公式可以看实验报告
switch name
    case "eqk"

        if (length(param) == 2) %平方指数核函数
            Distance = (x1.^2)' + (x2.^2) - 2 * x1' * x2; % L2范数
            result = param(2)^2 * exp(-0.5 * Distance / (param(1)^2));
        else
            disp("参数个数与核函数不匹配!");
            return;
        end

    case "pk"

        if (length(param) == 3) %L1范数周期核函数
            Distance = abs(x1' - x2); % L1范数
            result = Distance * pi / param(3);
            result = 2 * sin(result).^2 / (param(1))^2;
            result = param(2) * exp(-result);
        else
            disp("参数个数与核函数不匹配!");
            return;
        end
end
```

```

end

case "lpk"

    if (name == "lpk" && length(param) == 4) %混合周期核函数
        Distance1 = abs(x1' - x2); % L1范数
        Distance2 = (x1.^2)' + (x2.^2) - 2 * x1' * x2; % L2范数
        pk = Distance1 * pi / param(3);
        pk = 2 * sin(pk).^2 / (param(2)^2); % PK部分
        eqk = 1 ./ (2 * param(1)^2 * Distance2); % EQK部分
        result = param(4) * exp(-pk - eqk); % 混合
    else
        disp("参数个数与核函数不匹配!");
        return;
    end

otherwise
    disp("无效的核函数!");
    return;
end

end
end

```

- 优化方法展示

优化问题的目标是最小化

$l(\theta) = -\log P(y|X, \theta) = \frac{1}{2} \log |K + \sigma^2 I_n| + \frac{1}{2} y^T (K + \sigma^2 I_n)^{-1} y + \text{const}$ 。这是一个较为复杂的非凸问题，难以取得解析解，需要使用数值的方法解决，例如SGD，MCMC等。

由于时间限制，我并没有实现较为精巧的数值算法，而是采取了一种更为暴力的措施——参数空间遍历搜索，来防止优化问题陷入局部最小值。这个方法其实很简单：在一定的范围内随机生成多组参数(线性取值或者是对数取值)，每组参数分别使用MATLAB自带的优化函数收敛到局部最小值，然后对这些参数的优化结果进行择优录取。****在搜索的参数组数足够的情况下，整个函数的全局最小值大概率会成为某组参数的局部最小值，从而被择优录取所选择到。**

这种方法降低了实现难度，但是带来的就是计算复杂度的提升。假设核函数有着N个参数，每个参数有t个取值，那么需要遍历的参数组合就有 t^N 个。这种指数级的增长显然是无法承受的。因此，我和同学讨论后，提出了一种解决办法：我们不遍历所有 N^t 个参数组合，而是做出让步，生成每一个参数的t个取值之后，将其打乱，与另外参数的取值(也是打乱过的)直接进行组合，这样就把参数组合从 t^N 个降低到了t个，在可实现范围之内。

同时，在算法实现的过程中，我注意到了一个难点。在数据量过大，且引入的噪声不大的时候(噪声就是 $f(x^*)|y \sim N(K * (K + \sigma^2 I_n)^{-1}y, K^{**} - K * (K + \sigma^2 I_n)^{-1}(K^*)^T)$ 中的 σ)，矩阵 $K + \sigma^2 I_n$ 很容易变得奇异，导致其行列式的绝对值趋向于0或者是Inf，扰乱整个算法的执行。因此，我在实现的时候，针对这个矩阵的行列式做了特判：如果发现奇异值(0或者Inf)，则将这次优化的结果设置为Inf，标志着优化失败，从而避免了问题的奇异性对结果的影响。

参数优化由Train.m下的子函数Train实现。这个函数接受训练数据，优化初始值，参数遍历组数，参数上下界，参数采样方式(线性 or 对数采样)，以及核函数种类，完成参数的最终优化。其核心代码如下：

```
function [param] = Train(x_train, y_train, paramInit, argsnum, bound, logspace_flag, name)
% 采取暴力方法,取很多组初始值一个个优化
argsSearchTable = zeros(length(paramInit), argsnum);

for i = 1:length(bound) / 2 %生成大量随机参数
    lowerBound = bound(2 * i - 1); %下界
    upperBound = bound(2 * i); %上界

    if logspace_flag == 1
        logArray = linspace(log(lowerBound), log(upperBound), argsnum);
        tmpArray = exp(logArray);
    else
        tmpArray = linspace(lowerBound, upperBound, argsnum);
    end

    tmpArray = tmpArray(randperm(length(tmpArray))); %随机打乱
    argsSearchTable(i, :) = tmpArray;
end

%对生成的随机参数每个进行优化搜索
fmin = Inf; %初始化最小值变量

for i = 1:argsnum
    param0 = argsSearchTable(:, i); %优化初值
    H = @(param)KernelLoss(x_train, y_train, param, name); %构造优化用的函数指针
    [tmpargs, fval] = fminunc(H, param0); %优化问题

    if fval < fmin && fval > 0 %避免奇异情况
        fmin = fval;
        argsmin = tmpargs;
    end

end

disp(fmin)
param = argsmin;
end
```

3.运行结果展示

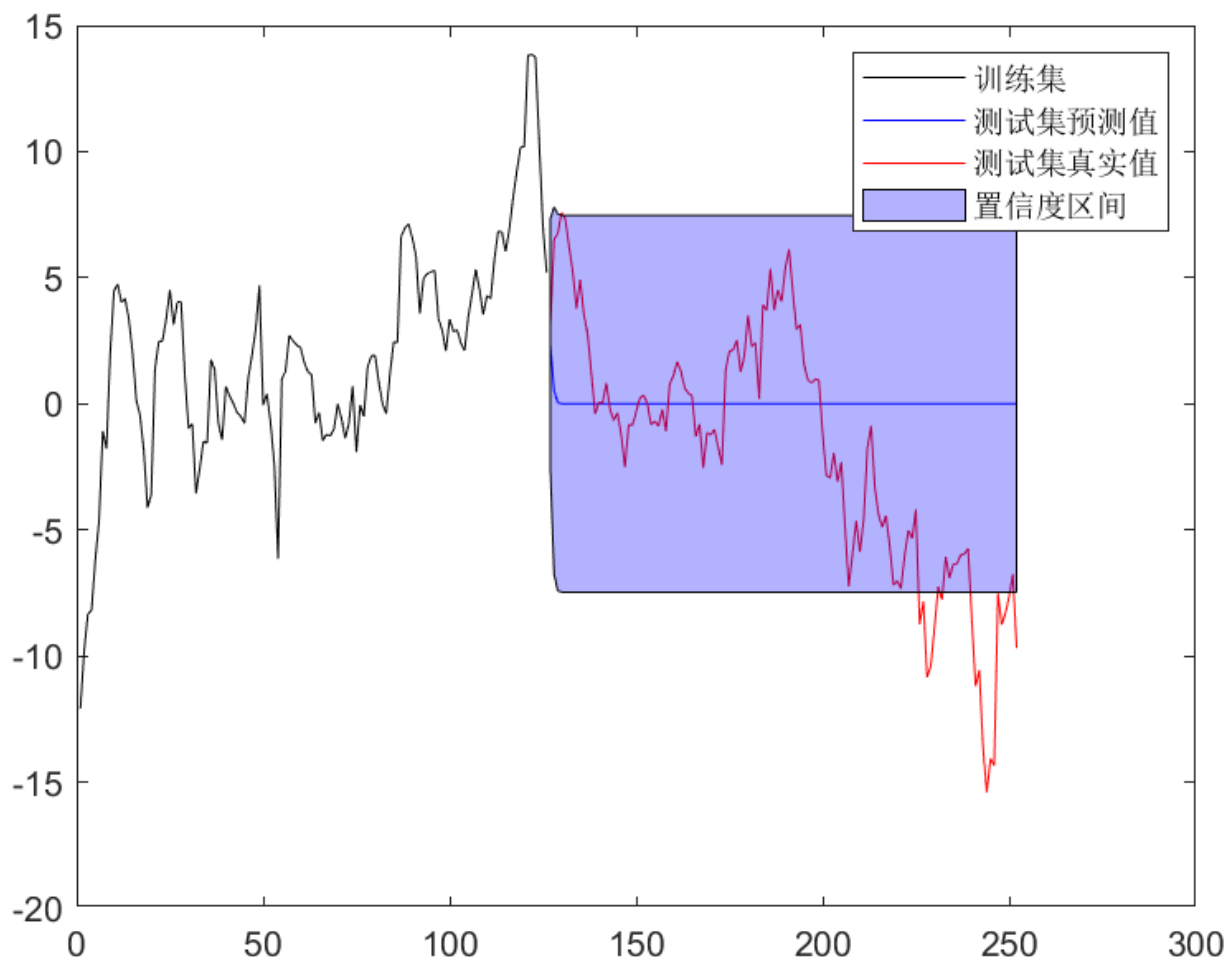
- 长时预测结果

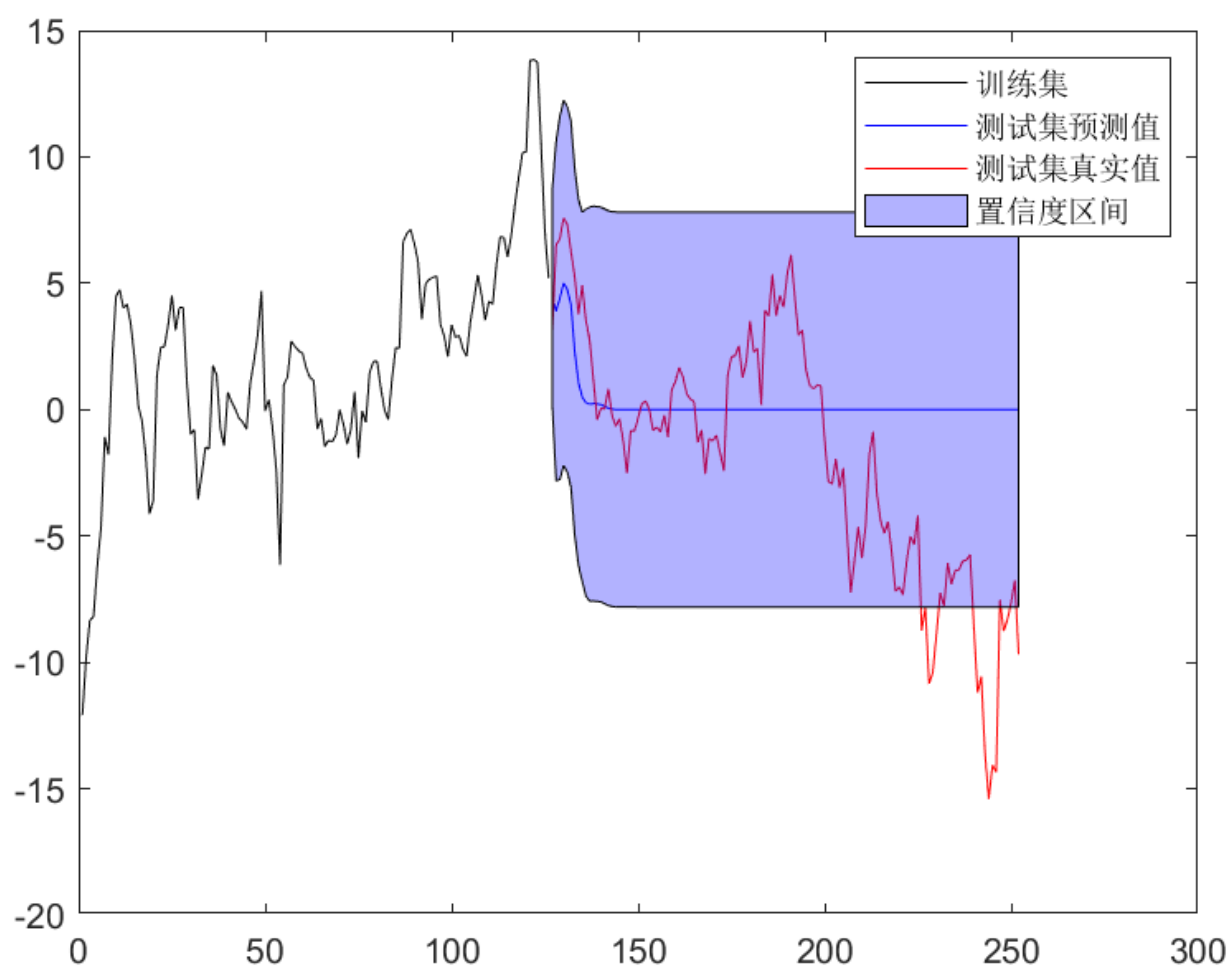
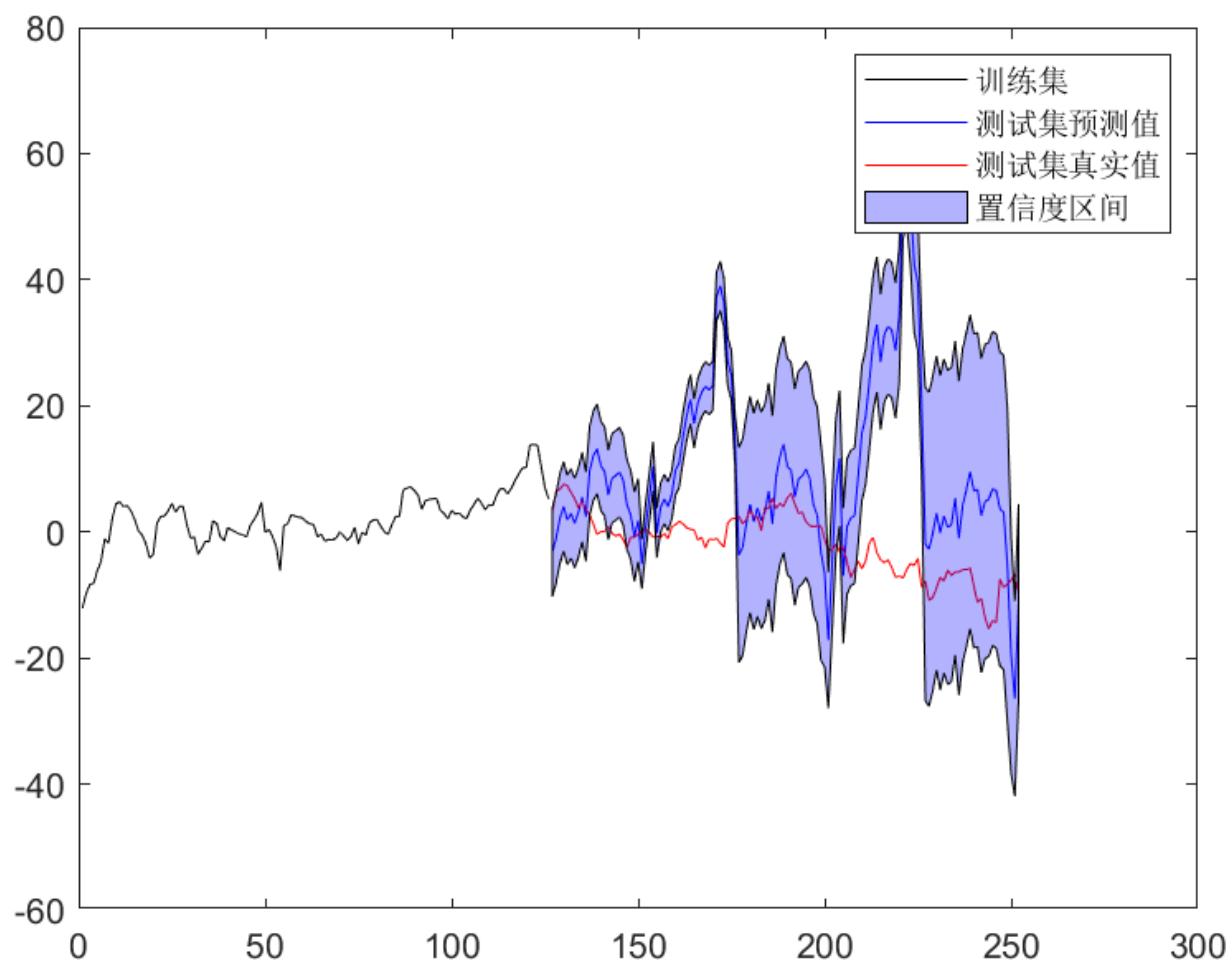
在代码完成后，我首先进行了长时预测的尝试。这种预测将整个股价-时间序列的前一部分列为训练数据，然后利用GPR统一产生剩下部分的预测值。由于一次预测覆盖了较长的时间，因此我认为这是GPR的长时预测。

长时预测的代码在**Predict.m**中的子函数**Predict**中实现，有详尽的备注，在此不再列举。

在数据方面，我选取了星巴克公司(股票代码SBUX)的1年股价-时间数据，将前50%的数据列为训练集，剩下的作为测试集，参数搜索组数选取100，选取方式为对数空间选取，在不同的核函数设置下进行了运行，得到的结果如下(三张图分别为EQK，PK，LPK三个核函数的运行结果)：

注：图中蓝色区域是高斯分布的**95%置信度区间**





由上面三张图可以看见，高斯过程回归在长时预测上的性能并不是很好。使用EQK和LPK函数时，预测值会回归到数据的均值0(我在最开始做了归一化)，并且得到的方差也较大，导致置信度区间比较宽泛。LPK核函数在7天内的预测效果较好，而EQK函数完全没法预测未来的股价。

而在使用PK函数的时候，预测值没有回归均值，呈现周期性，这与PK函数的特性相同。但是置信度区间交窄，预测值与真实值偏离较为严重，并不是很良好的预测。

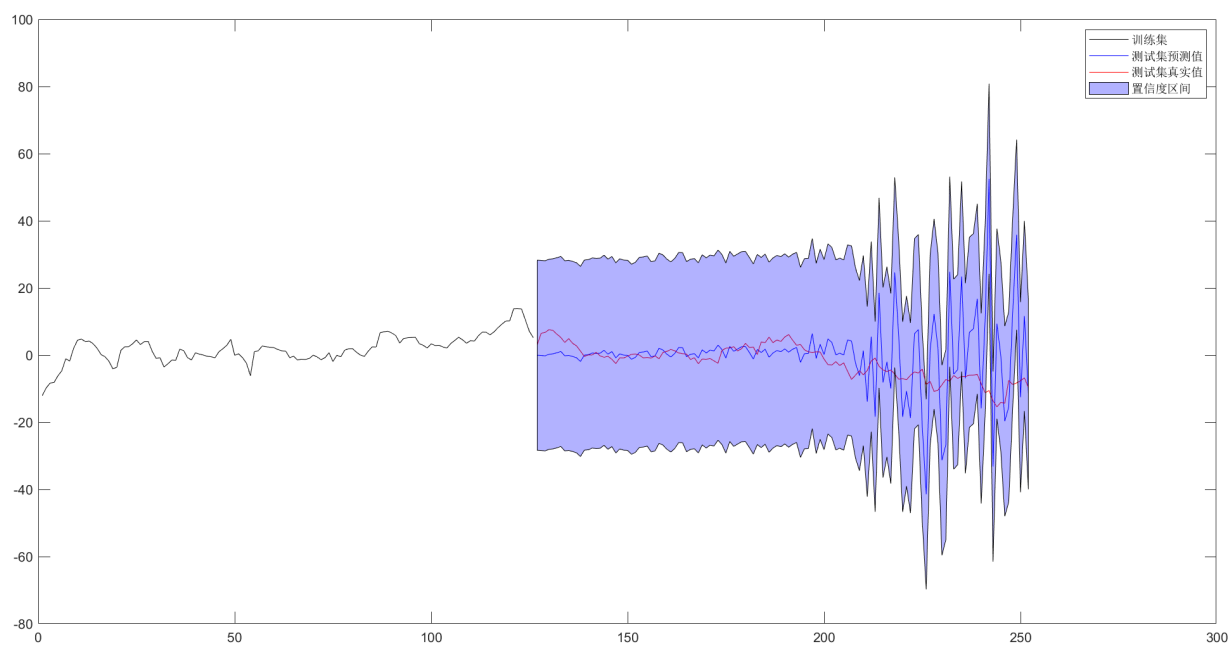
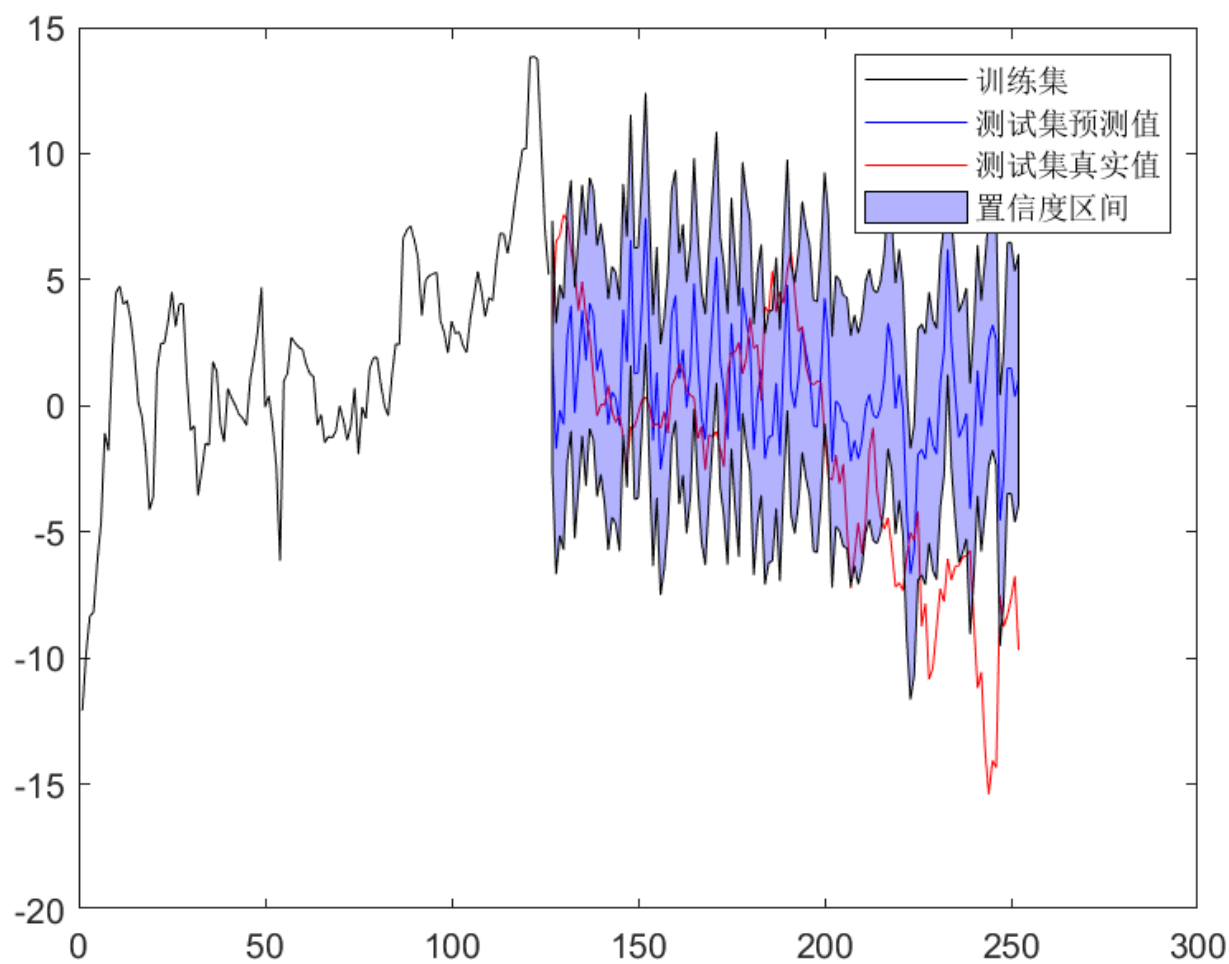
在我看来，这是因为核函数是基于L1/L2范数的负指数构建的。在长时预测任务中，两个向量之间的L1/L2范数都非常小，从而导致协方差矩阵 K^* 中的非对角线值几乎都为0，也就是先前的训练数据与之后的预测数据毫无关系，这导致依靠前期数据进行预测变成了几乎无法实现的任务。

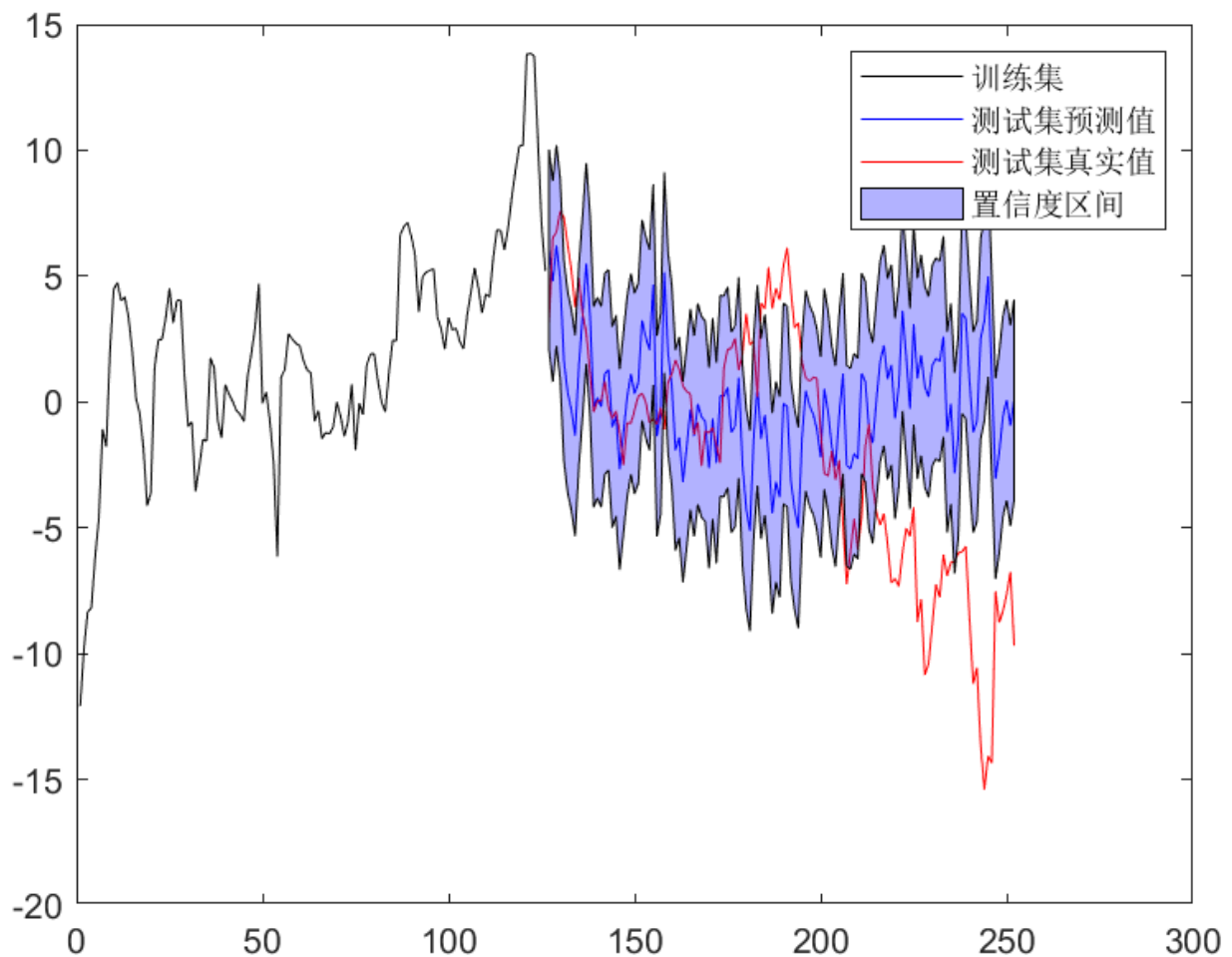
- 短时迭代预测结果

- 既然不能依靠长时预测进行一劳永逸的预测，在于同学讨论之后，我将目光投向了短时预测上。短时预测的核心思想是：我不是一次性产生所有的预测数据，而是每一轮迭代都产生1天的数据，然后根据这一天的预测数据更新训练集。如此一天天迭代，从而产生之后所需要预测的数据。

依照这个思想，我在ShortPredict.m中实现了函数ShortPredict，来进行迭代式的短时高斯过程回归，从而产生一段长时间的预测数据。在一天的预测结束后，我利用当天产生的均值和方差，遵循高斯分布地随机取一个数作为这一天的(x,y)添加进入训练集，同时删除训练集的第一个元素。

保持参数不变，改变三个核函数，我对这种短时迭代预测方式进行了测试与可视化，得到的结果如下(三张图分别为EQK，PK，LPK三个核函数的运行结果)：





比较这些图与长时预测的图，可以发现较为良好的改进。无论是哪个核函数，都没有再出现“预测值回归数据平均值”的现象，这说明这种方法能够解决长时预测中的核函数消失问题。

同时，相较于长时预测，核函数为EQK或LPK时，置信区间都变小了，且置信区间在 $x < 200$ 的时候基本能覆盖测试集的真实数据。这说明短时迭代预测也有着比较好的预测性能。

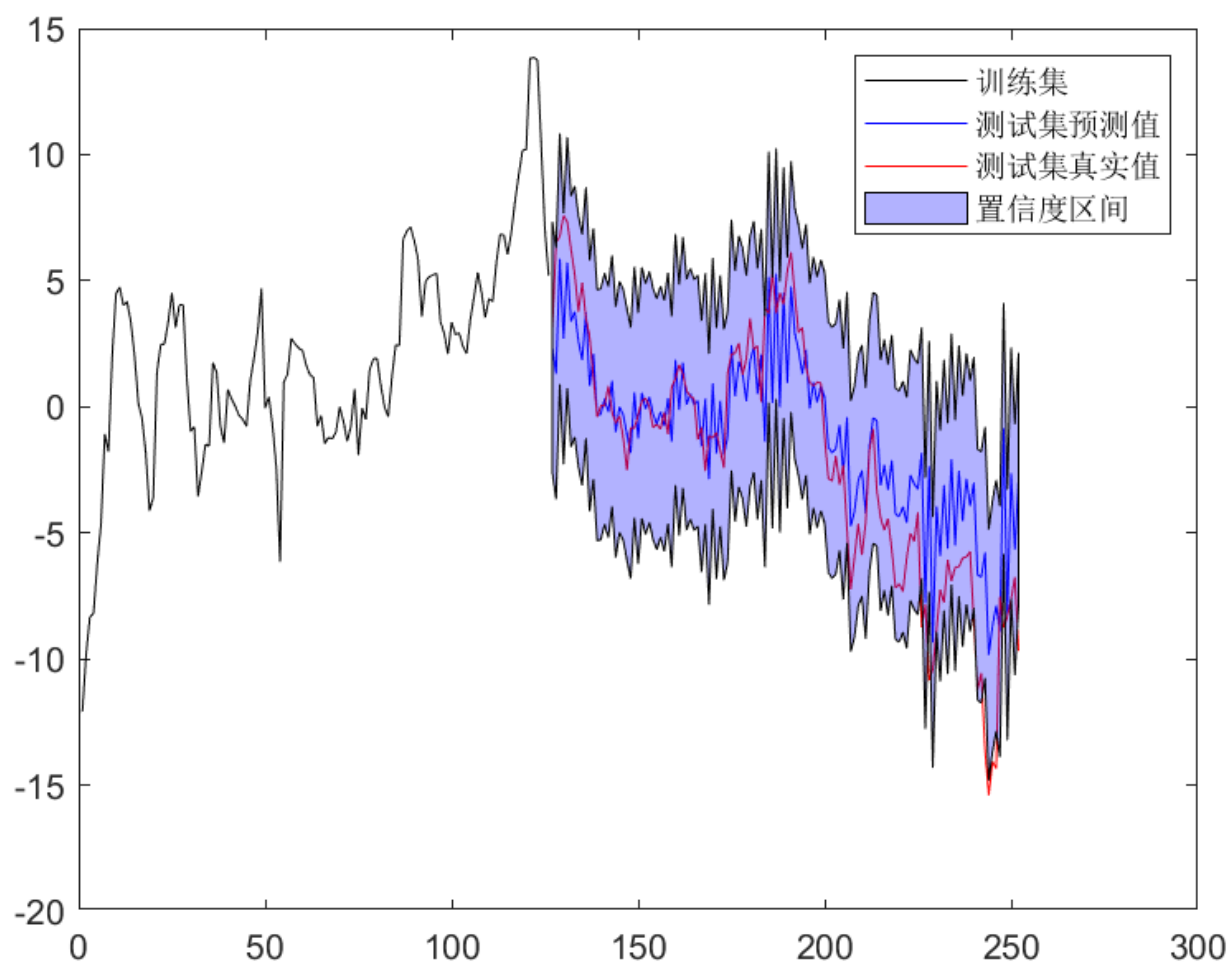
但是核函数为PK时，预测值在 $x > 200$ 的时候发生了较大的振荡，核函数为EQK或者LPK时，预测值在 $x > 200$ 的时候也都发生了程度不一的偏离。这是很好理解的，因为迭代是预测是将上一个预测结果作为下一个预测结果的基础，这就造成了误差的累积，随着时间越来越长，预测结果会越来越偏离真实值。

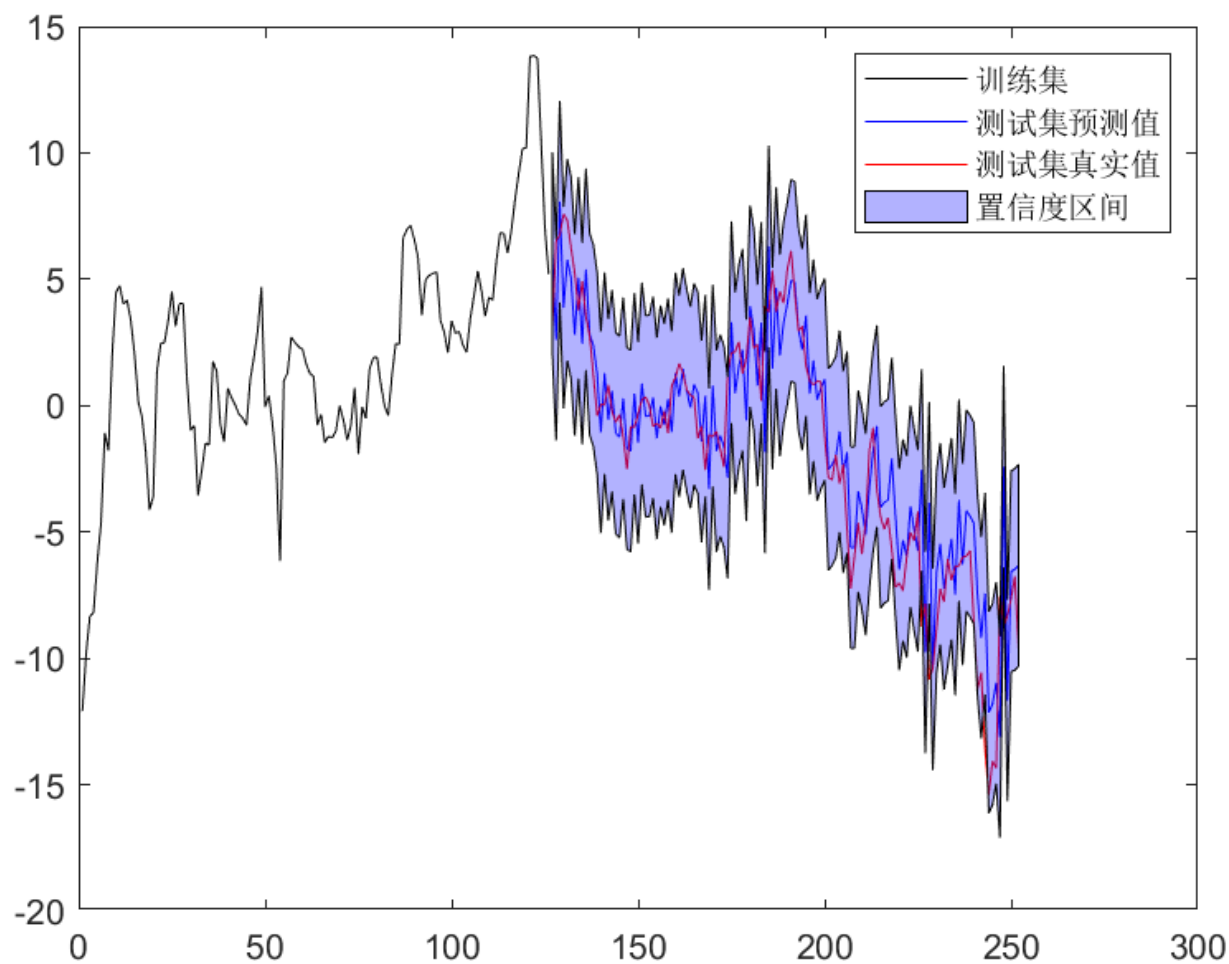
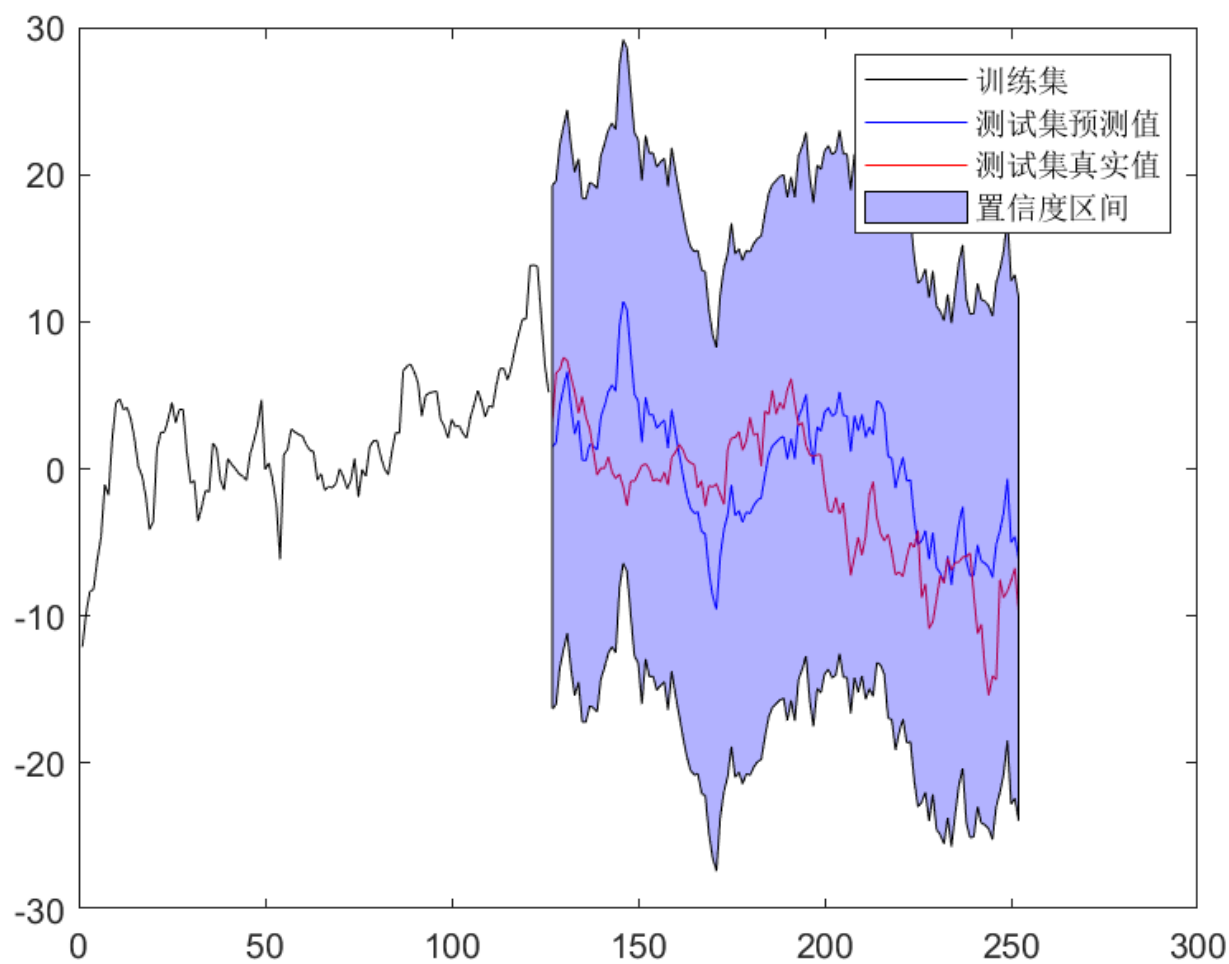
- 短时预测结果

在完成了上面的短时迭代预测之后，我又想到，如果我们加强条件，利用需要预测时间之前的真实数据作为训练集，预测当天的股价会怎么样呢？

秉持着这个想法，我修改了ShortPredict函数，使其兼容这种与短时迭代预测不同的“短时预测”。在一天的预测结束后，删除训练集的第一组数据，但是加入训练集的并不是根据当天预测值所取的随机数，而是当天的真实值。这就相当于对每个预测点都做短时GPR预测。

保持参数不变，改变三个核函数，我对这种短时迭代预测方式进行了测试与可视化，得到的结果如下(三张图分别为EQK，PK，LPK三个核函数的运行结果)：



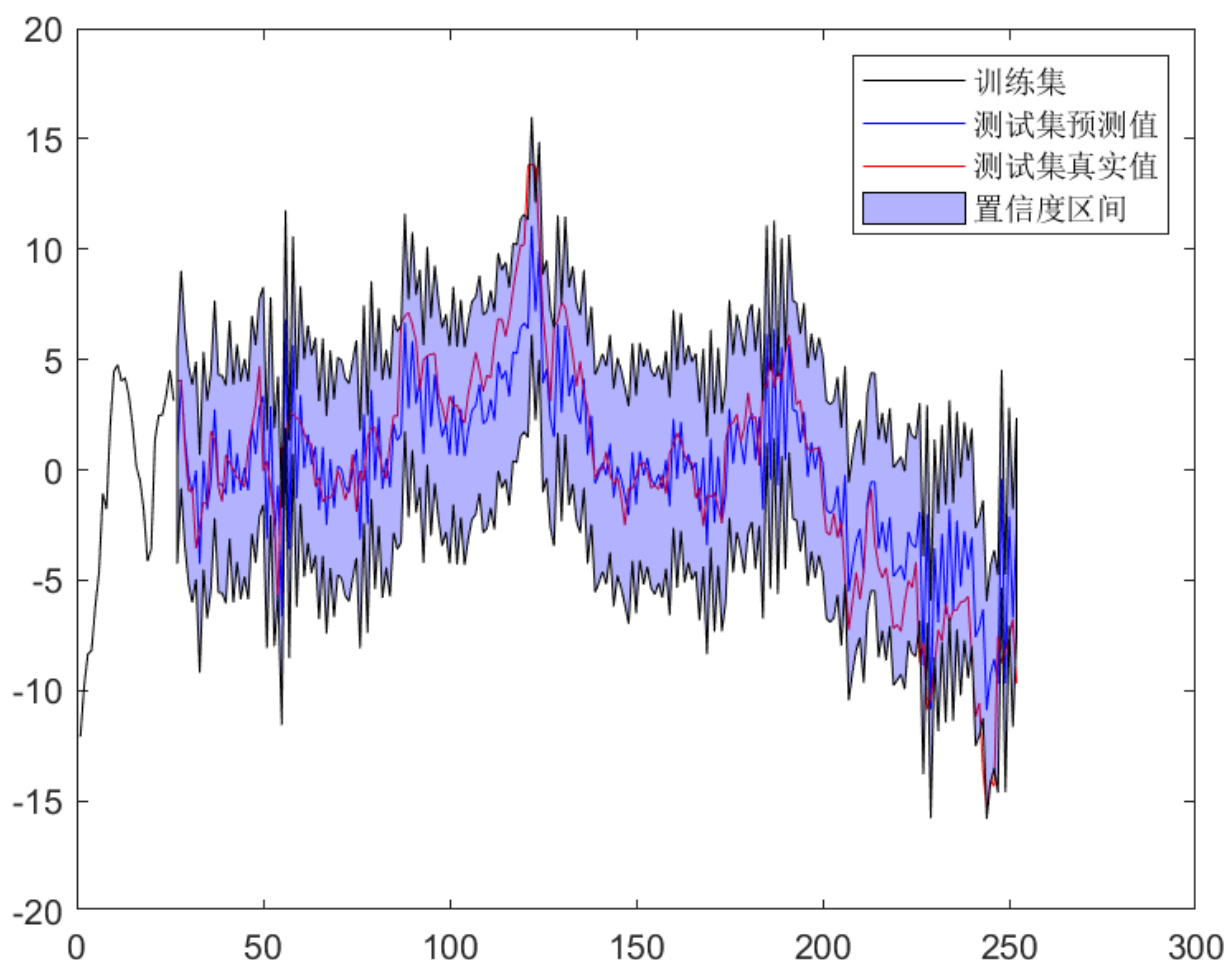


比较这三张图与之前的六张图，可以看到短时预测相比于短时迭代预测和长时预测，预测的准确度和置信度都有了飞跃。短时预测基本能完成准确预测股价-时间走势，以及股价本身。

当核函数是EQK或LPK时，可以看到GPR产生的预测值和真实值相差无几，可以说接近完美的预测。

而当核函数是PK时，置信度区间较宽，预测值和真实值的区别也较大，我猜测这可能是PK核函数的短时周期特性所导致的。

事实上，即使将训练集的比例从0.5缩小至0.1，这种短时预测也能给出优秀的预测结果：



但是，这种预测也有其弱点：因为每一次短时预测都需要直至当天之前的真实股价-时间数据，因此这种短时预测只能提前一天产生预测值，在时间层面上可能有比较大的缺点。

4.横向对比

本次实验所对比的外国学生的Project是参考文献[3]。

在[3]中，作者首先针对股票的涨跌进行预测。他将股票涨跌量化为二进制变量，然后使用GPR分类进行预测，但是这种分类在训练集较大的时候效果不佳(**poor prediction**)。这是因为股票涨跌作为离散型随机变量(过程)，与预测连续值的GPR难以契合，并且股票涨跌和投硬币一样，几乎都是50%的概率，难以直接对其性质进行预测。

之后，作者尝试了利用GPR进行长时间预测，和我一样遇到了预测回归平均值0的结果(**regression only predicted the mean value 0**)。

但是我利用GPR在短时预测上的准确性与优越性，给出了短时迭代预测和短时预测两种解决方案(具体见上一部分)，这是外国学生的Project中所没有涉及的，更深层次的部分。这种短时预测不会产生平均值0的预测，也更加准确，更加可信，这就是我所提出的方法的优势与特点。

5.心得总结

在本次实验的开始，我对高斯过程回归(GPR)一无所知。但是在实验指导书的指引下，我从阅读文献开始，一步步理解了GPR的原理，编写了GPR回归和参数优化的MATLAB代码，最终完成了性能优秀的股价-时间预测任务(短期/长期)，并且做了大量对照实验，给出了可视化结果。

在这个过程中，我受益良多。我不仅加深了对高斯过程认识，意识到了高斯过程的本质之强大与应用值广泛。同时，这也是全新的，从文章和课件走向代码成果的体验，就像是一次小小的“论文复现”一样，不仅提升了我英文专业文献阅读的能力，在我之后的科研之路上也能为我带来巨大的便利。

总而言之，感谢老师与助教为这个实验，以及《概率论与随机过程》这门课程的付出！同时我也要感谢康书诚，于子涵等同学在完成作业中与我共同讨论，为我最终代码的实现提供了不少参考意见。

6.参考文献

[1]M.Ebden, " Gaussian Processes for Regression An Quick Introduction".

[2]H.M.Wallace, "Introduction to Gaussian Process Regression"

[3]M.T. Farrell, et al, "Gaussian Process Regression Models for Predicting Stock Trends".

[4]SCHULZ E, SPEEKENBRINK M, KRAUSE A. A tutorial on gaussian process regression: Modelling, exploring, and exploiting functions[J]. Journal of Mathematical Psychology, 2018, 85:1-16.

注：代码都存放在根目录文件夹下，清单如下：

main.m	主函数，运行这个文件就能得到结果与图
DataProcess.m	数据处理函数，读取\data中的数据并且进行切片
KernelFunc.m	核函数的实现
KernelLoss.m	核函数Loss的计算，就是那个负对数
Predict.m	GPR长时预测函数
ShortPredict.m	GPR短时预测函数
Train.m	参数优化搜索函数
DrowPlot.m	画图函数